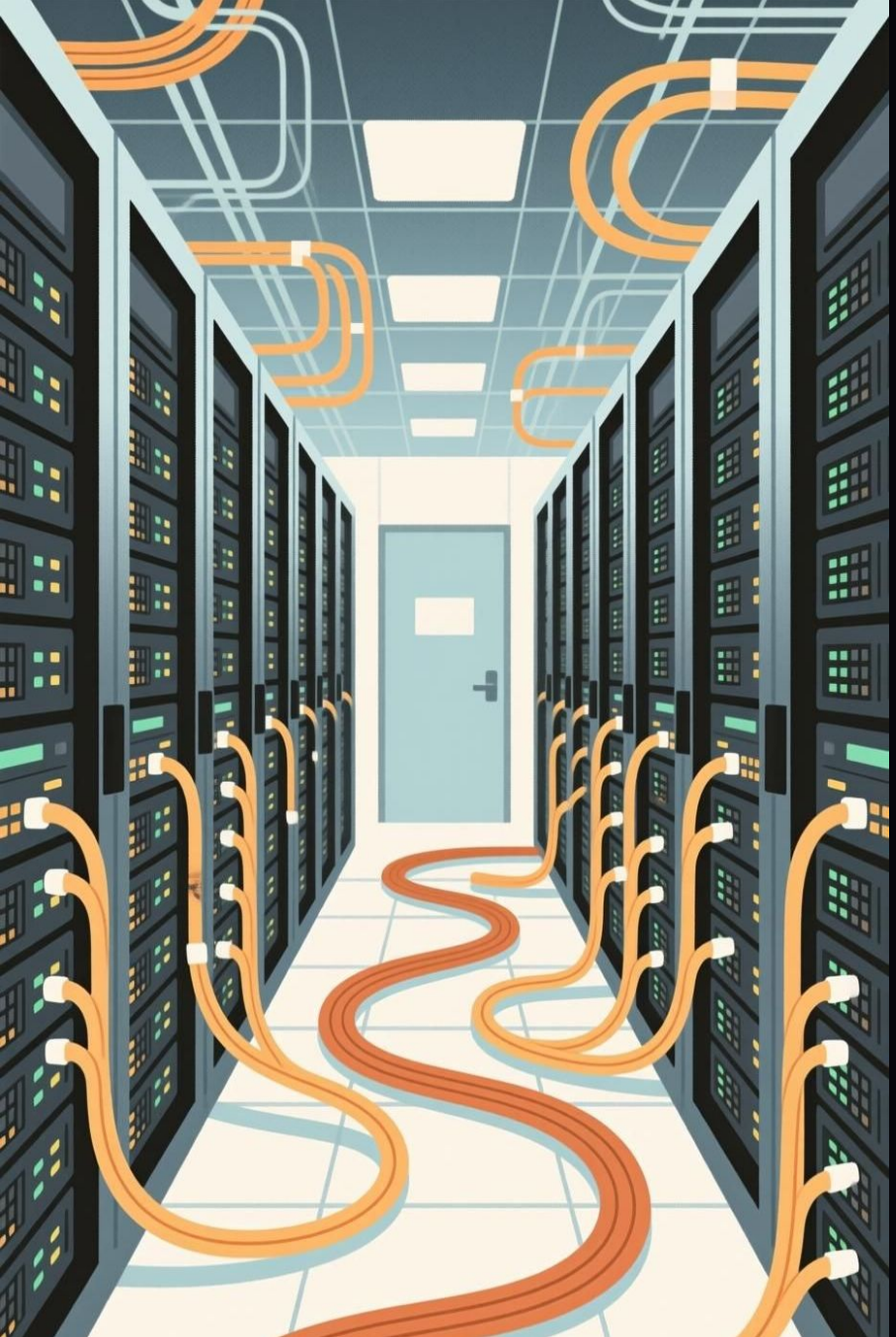


MCP Network Optimizer

Intelligent, Safe, and Autonomous Network Optimization via Model Context Protocol

What is MCP (Model Context Protocol)?

MCP (Model Context Protocol) is an open protocol developed by Anthropic to standardize how applications (like Claude) connect to and use external data sources and tools.

How it Works

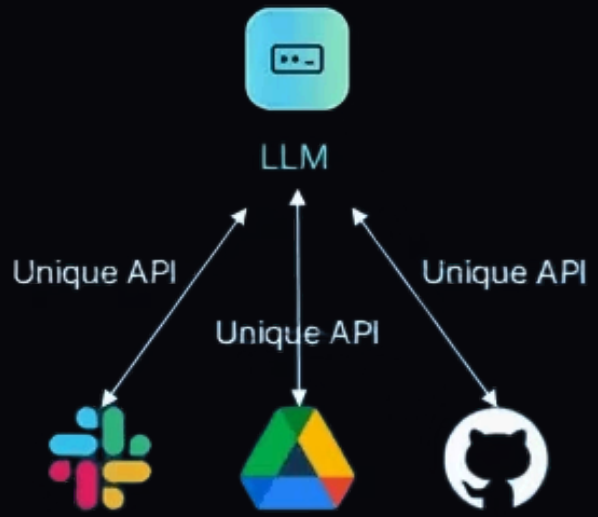
1. **Connection & Discovery**
   - Client connects to MCP server via STDIO/HTTP
   - Server announces available tools and resources
2. **User Request**
   - User asks natural language question
   - LLM analyzes intent and identifies needed tools
3. **Tool Execution**
   - Client calls appropriate tool via MCP protocol
   - Server executes the action and returns results
4. **Response Generation**
   - LLM incorporates tool results into context
   - Delivers informed, actionable response

Key Benefits

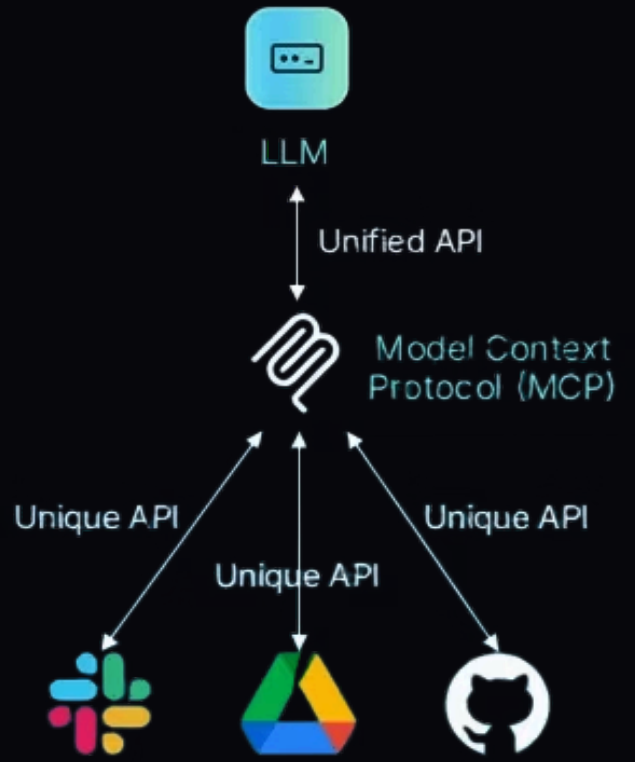
- **Standardized Tool Access:** Provides a universal way for AI to access and utilize diverse external functionalities.
- **Enhanced Security:** Implements controlled interfaces to safeguard systems from unauthorized or erroneous AI operations.
- **Broad Extensibility:** Easily integrates new tools and data sources, future-proofing AI capabilities and applications.

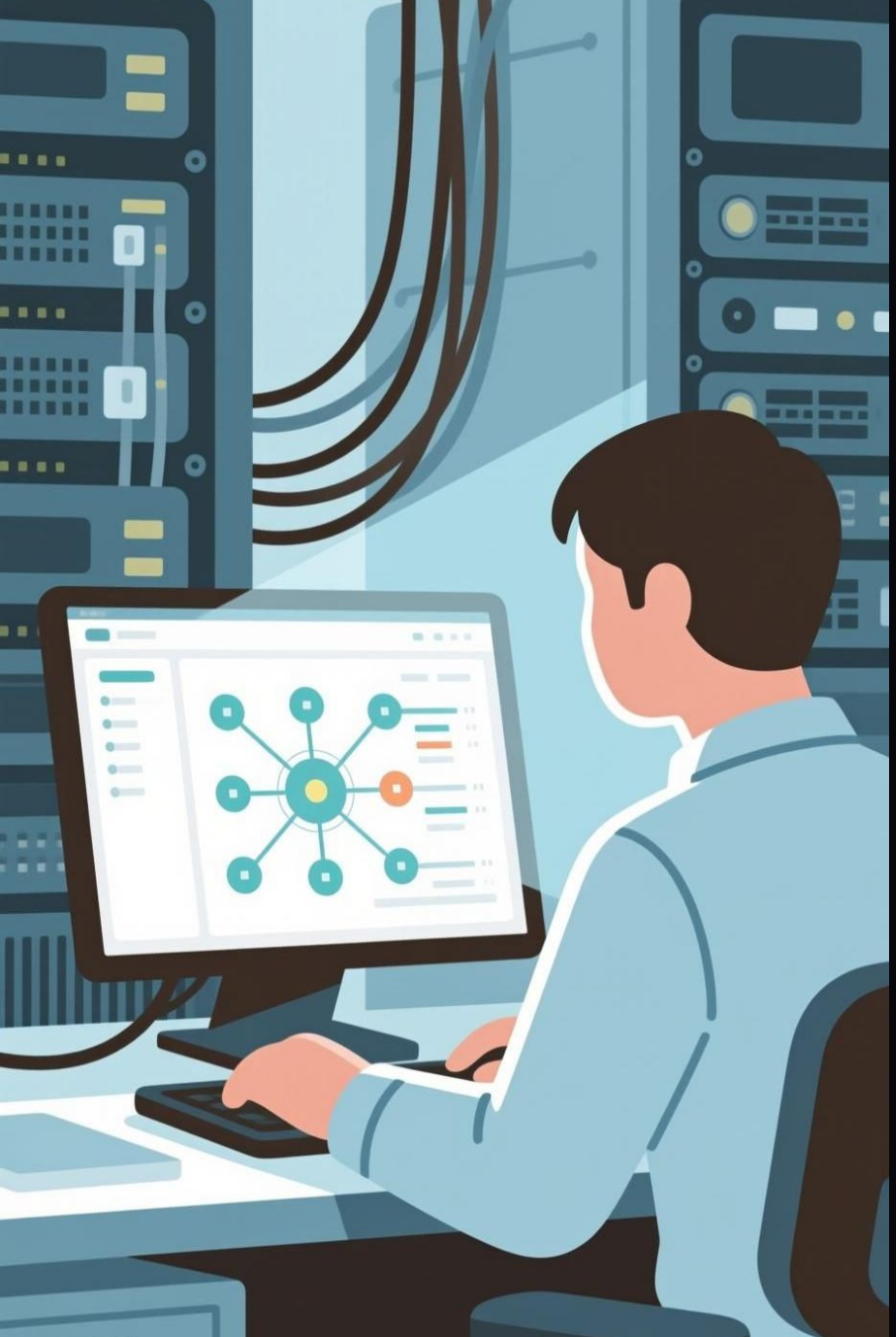
descope

Before MCP



After MCP





The Network Optimization Challenge

Three Core Problems:

1 Complex Configuration

- High-level goals require error-prone low-level commands.
- Requires deep kernel expertise.

2 Unsafe Operations

- Manual changes risk outages & drift.
- No rollback or consistency at scale.

3 No Smart Automation

- AI agents lack safe interfaces to optimize networks.
- Can not validate or adapt to dynamic workloads.

Architecture: The Five-Stage Pipeline

MCP Network Optimizer transforms high-level optimization goals into safe, validated system changes through a deterministic workflow:

1

Discover

Gather current network configuration and performance metrics via 30+ safe introspection tools like `ip` and `dig`.

2

Plan

Define optimization goals declaratively using type-safe Pydantic schemas and profiles.

3

Validate

Enforce policies, limits, and safety constraints against declarative changes.

4

Render

Translate abstract specifications into concrete Linux commands (`sysctl`, `tc`, `nftables`).

5

Apply

Execute atomically with automatic checkpoint/rollback on failure and comprehensive audit logging.

Configuration Coverage: 29 Cards Across Three Domains

Comprehensive Linux networking stack control through specialized configuration cards, each mapping to executable system commands with validation and safety constraints:

Sysctl(System Control Parameters) (16 Cards)

TCP/IP Tuning: Congestion control (BBR, CUBIC), buffer management (rmem/wmem), window scaling, FastOpen, connection timeouts, netdev budget, and queueing discipline defaults.

Traffic Control (7 Cards)

QoS & Shaping: Queue disciplines (fq, fq_codel, cake, HTB), network emulation (`netem`) for latency/loss, and bandwidth shaping policies.

Firewall/Security (6 Cards)

Protection & Limits: Per-IP connection limiting, packet rate limiting, connection tracking (`conntrack`), DSCP QoS marking, and NAT rule configuration.

Five Optimization Profiles for Diverse Workloads

Research-backed profiles activate curated configuration card combinations, each optimized for specific network performance goals and workload characteristics:

🎮 Gaming	📺 Streaming	📞 Video Calls	📦 Bulk Transfer	🖥️ Server
Ultra-low latency	Max throughput	Balanced latency/BW	Absolute max BW	High concurrency + security
BBR, low jitter	BBR, Buffers	DSCP QoS marking	Buffers, BDP	SYN cookies, rate limit
13 cards active	12 cards active	13 cards active	12 cards active	17 cards active

Safety Architecture

Command Allowlisting

- Only pre-approved binaries can be executed.
- No shell injection possible; all commands use argv arrays, never shell strings.

Pydantic Schema Validation

- Type checking, range validation (e.g., 16-128MB buffers)
- Enum constraints (e.g., congestion control options)

Policy Enforcement

- All changes validated against policy/limits.yaml, checking buffer sizes, rate limits, port ranges, and connection maximums.
- No deviation from safety thresholds permitted.

Checkpoint/Rollback

- Atomic snapshots captured before changes
- Any failure triggers automatic restoration to known-good stateManual rollback supported via checkpoint IDs for incident recovery.

Audit Logging

- Complete execution history records all commands, timestamps, success/failure status, change rationale, and checkpoint IDs.
- Full traceability for compliance and debugging.

MCP Discovery Tools (Stage 1)

- **Network Interfaces:** Tools like `ip`, `ethtool`, and `nmcli` show device setups, link status, and active connections.
- **Routing:** `ip route`, `ip neigh`, and `arp` reveal routes, neighbors, and MAC mappings.
- **DNS:** `resolvectl`, `dig`, and `nslookup` help diagnose domain and name server issues.
- **Performance:** `ping`, `traceroute`, and `tracepath` test connectivity and locate network delays.
- **QoS/Firewall:** `tc qdisc show` and `nft list ruleset` display traffic control and firewall rules.
- **Sockets:** `ss` lists active connections and ports for service monitoring.

Planning & Validation Pipeline (Stages 2-3)

Stage 2: Plan

- Define optimization goals declaratively using type-safe Pydantic schemas and profiles.
- Goals translate into specific configuration specs via five research-backed optimization profiles, each tuned for distinct network performance and workload types:
  - **Gaming**: Ultra-low latency
  - **Streaming**: Max throughput
  - **Video Calls**: Balanced latency/bandwidth
  - **Bulk Transfer**: Max bandwidth
  - **Server**: High concurrency + security
- This schema-based approach ensures type safety and structural validity via Pydantic before applying optimizations.

Stage 3: Validate

- Before applying any configuration changes, strict checks ensure everything is safe and compliant.
- Pydantic models verify data types, valid ranges, and allowed values to prevent bad inputs.
- Each change is checked against `policy/limits.yaml` to confirm it stays within approved buffer sizes, rate limits, and port or connection ranges.

Render & Apply Pipeline (Stages 4-5)

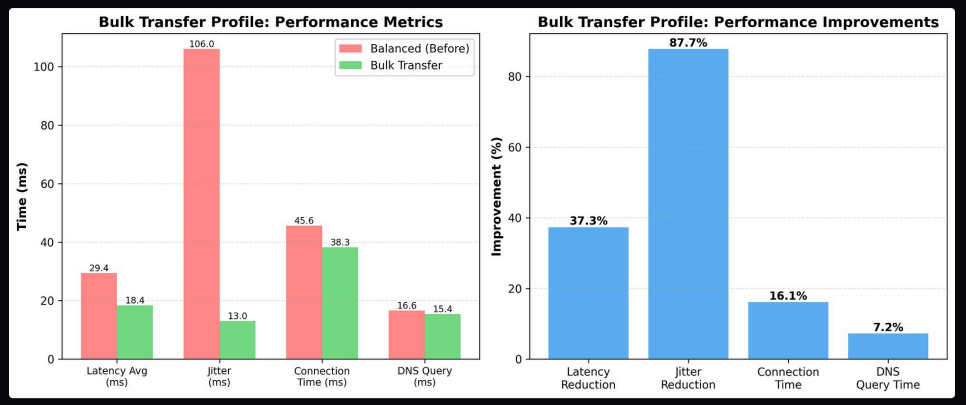
Stage 4: Render

- Translate abstract specifications into concrete Linux commands. Abstract configurations become executable commands for various network settings:
  - **sysctl:** Kernel parameter adjustments.
  - **tc (traffic control):** Shaping and scheduling network traffic.
  - **nftables:** Advanced firewall and packet filtering rules.
- Command safety is ensured through allowlisting and argv arrays. Only pre-approved binaries can execute.
- No shell injection is possible—all commands use argv arrays, never shell strings.

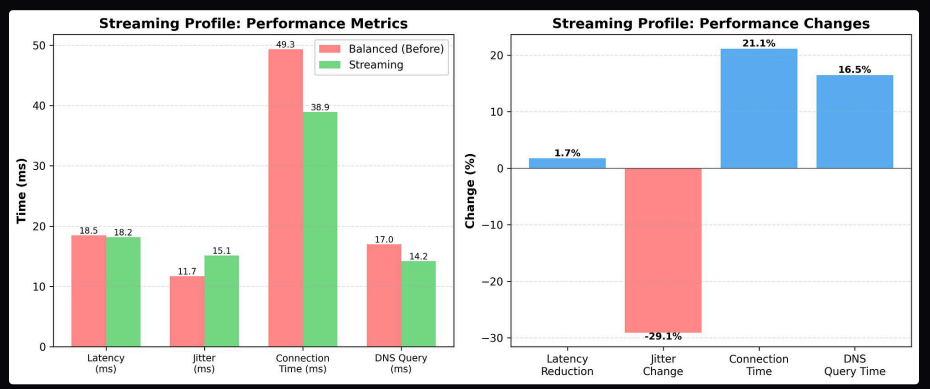
Stage 5: Apply

- **Base Checkpoint:** Create a base checkpoint.
- **Atomic Execution:** Execute appropriate commands to configure the network parameters.
- **Rollback:** In case of error, or if asked by the user to rollback, the saved checkpoint can be used switch to it.

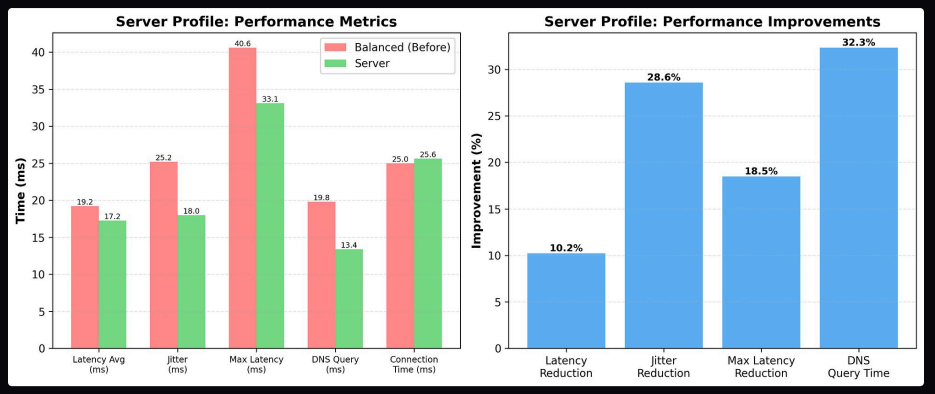
Results



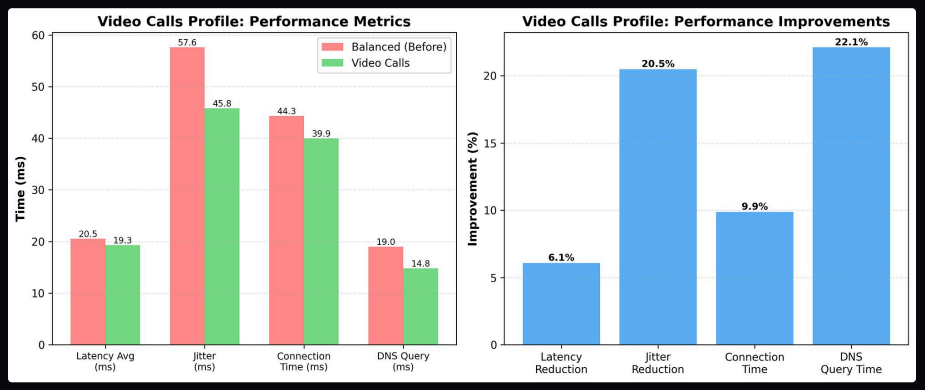
Bulk Transfer Profile



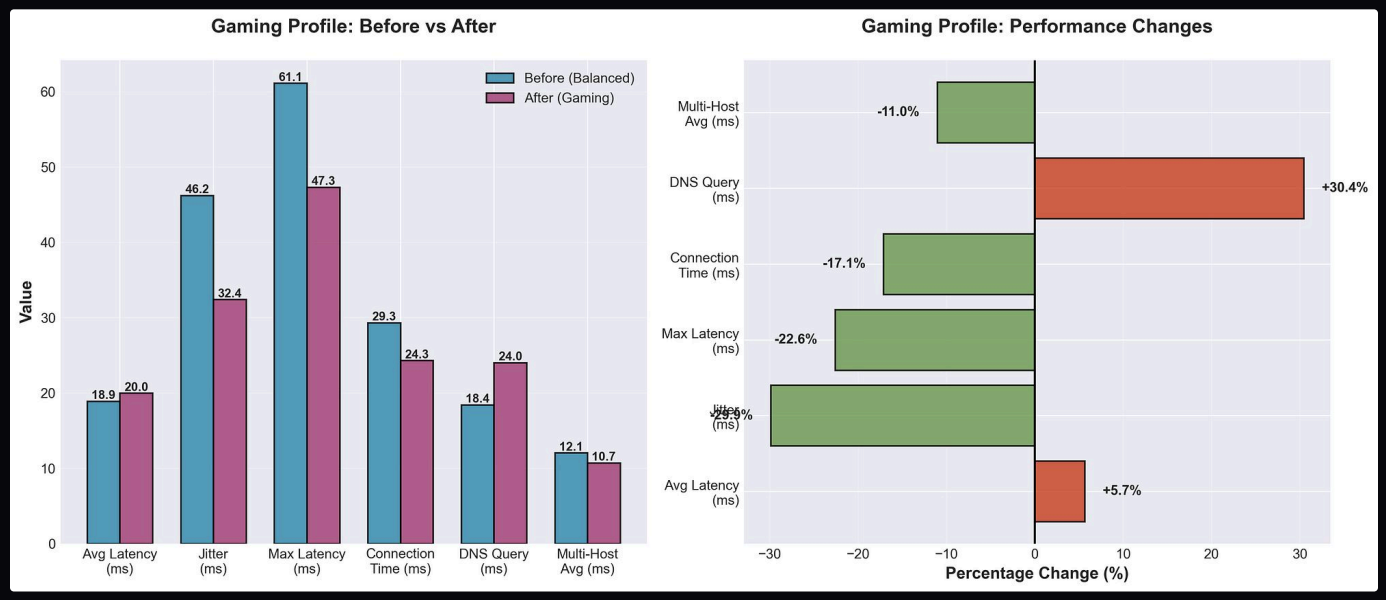
Streaming Profile



Server Profile



Video Calls Profile



Gaming Profile